

Produto e arquitetura

App Android e iOS em Kotlin Multiplatform + Compose Multiplatform · MVVM + Clean Architecture · Tiago Benaías Pompeo · outubro de 2026
Repositório: <https://github.com/tiagobpompeo/MiboSmartCase>

Problema e público

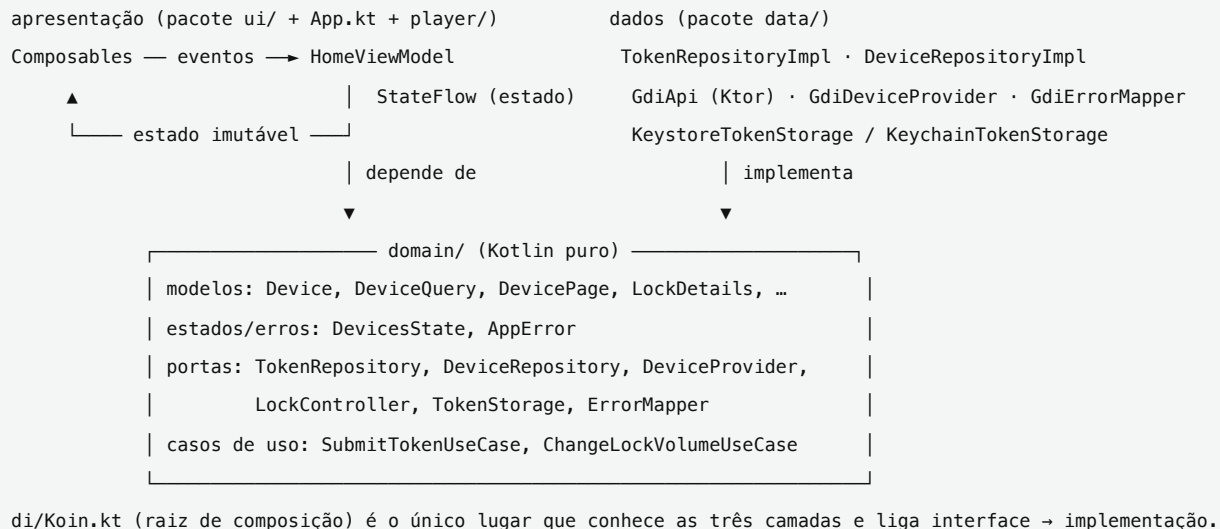
Quem tem câmeras, fechadura e hub Intelbras quer, num só app, ver a câmera ao vivo e saber se a porta está fechada, sem depender do app do fabricante para cada integração. A API GDI abre a plataforma Casa Inteligente para parceiros; este app é um cliente de parceiro: autentica com o token temporário da conta, lista os dispositivos e opera câmera e fechadura com a mesma aparência do app oficial. O público é o morador (uso diário, telas claras, retorno em cada estado) e, do lado técnico, o integrador que precisa acrescentar novos parceiros sem reescrever a UI.

Requisitos e estado

RF	Entrega	Estado
RF01	Tela de token: campo mascarado com mostrar/ocultar; "Entrar" só com token plausível; gravado cifrado só depois que a 1. ^a página responde; reabrir vai direto à Home	Feito
RF02	Lista na ordem da API com nome, tipo inferido, online, "via hub" e "compartilhado"	Feito
RF03	Vídeo ao vivo com %, etapa, "AO VIVO" e tempo do 1.º quadro; libera player e sessão ao sair; lentes da Dual; Android e iOS	Feito
RF04	Mensagens distintas para token ausente, inválido (401), expirado (403), sem rede, timeout, erro do servidor e lista vazia por filtro; detalhe técnico só em debug	Feito
RF05	Fechadura: estado, bateria, online, abertura remota; segurar 600 ms → confirmação → comando → releitura	Feito sem execução física (o comando nunca foi enviado ao equipamento real)
RF06	Volume Mudo/Baixo/Médio/Alto; erro de leitura honesto com escrita habilitada; grava e relê	Feito, com limitação da API: a GDI responde 500 à leitura do volume desta fechadura (pedido conferido, ver "Contrato real × Swagger"); a escrita nunca foi enviada ao equipamento real
RF07	Filtro Todos/Vinculados/Compartilhados no servidor; recomeça na página 1; resposta de filtro antigo descartada	Feito
RF08	Itens por página 2/5/10/50, "Carregar mais", "Sem mais informações", contagem; erro ao carregar mais mantém a lista	Feito
RF09	Histórico com 10 eventos, "Ver mais" com 30; vazio e erro com textos diferentes	Feito

O roteiro de 19 passos (RF01–RF09) foi executado no Android físico com a API real em 05/10/2026, com comparação lado a lado com as capturas de referência; no iPhone físico, token vencido, Home e vídeo ao vivo foram conferidos com a API real em 06/10/2026 (fechadura e hub, só no simulador). Pendentes: modo avião no aparelho e fechadura e hub no iPhone físico (detalhes na seção "Verificação em aparelho" do README do repositório).

Camadas



Só `data/gdi` conhece a GDI (rotas, nomes de campo, códigos de erro, formato de data); o que sai de lá já é modelo do domínio, com textos prontos para exibir. Toda falha vira `AppError` (selado, `when` exaustivo) num único lugar, o `GdiErrorMapper`. A UI recebe só interfaces do domínio por injeção.

Múltiplos parceiros

`DeviceProvider` é a porta de um parceiro (listar página, vídeo, firmware, online e, opcionalmente, um `LockController`). A GDI é um provider (`GdiDeviceProvider`). Um parceiro novo é um `single<DeviceProvider>(named("..."))` a mais no Koin: o `DeviceRepositoryImpl` recebe a lista com `getAll()`, pede a mesma página a todos, junta o resultado e encaminha cada operação ao parceiro dono do dispositivo pelo `providerId`. UI, `ViewModel` e repositório não mudam.

Modularização

Hoje: um módulo Gradle `shared` (mais `androidApp` e `iosApp`), com fronteiras por pacote (`ui` → `domain` ← `data`) conferidas por três buscas de `import` que têm de vir vazias: `domain` sem Ktor, Koin, `Compose`, `androidx`, `kotlinx.serialization` nem outras camadas; `ui`, `player` e `App.kt` sem `data` nem Ktor; `HomeViewModel` só com imports de `domain`. Para o tamanho do case isso mantém o build simples. Quando o app crescer, a divisão prevista é:

Módulo	Conteúdo
<code>:core:domain</code>	Kotlin puro: modelos, estados, <code>AppError</code> , portas e casos de uso
<code>:data:gdi</code>	cliente, parser e mapeador de erro da GDI; registra seu <code>DeviceProvider</code> no Koin (um módulo por parceiro)
<code>:feature:home</code>	token, Home e Definições
<code>:feature:camera</code>	tela de câmera e player
<code>:feature:lock</code>	fechadura e hub

`androidApp` e `iosApp` passam a depender só das features, e cada feature ganha o seu `ViewModel` no lugar do `HomeViewModel` único.

Interoperabilidade com Java

`commonMain` não compila Java, e uma regra escrita em Java teria de ser duplicada para o iOS; por isso as regras de negócio são Kotlin. A interop com Java acontece na borda Android, nos dois sentidos:

- **Java chamando Kotlin** (androidApp): CasaApplication.java inicia o Koin chamando a função de topo `initKoin` (no bytecode, o estático `KoinKt.initKoin`), passa o tipo-função `KoinApplication.()` -> `Unit` como `Function1` que devolve `Unit.INSTANCE` e chama a extensão `androidContext` como estático que recebe o receptor no primeiro argumento. `TokenFormatJavaTest.java` (JUnit 4) chama a regra do token como `TokenFormat.mask(...)`, graças ao `@JvmStatic` no object Kotlin.
- **Kotlin chamando Java** (androidMain): `java.security.KeyStore` e `javax.crypto` (armazenamento do token), `java.time` (faixa da semana), `java.util.concurrent` (thread de controle do libVLC), a API Java do libVLC (`org.videolan.libvlc`, JNI por baixo) e `android.util.Base64`. Em `commonMain`, `kotlinx.io.IOException` é typealias de `java.io.IOException` na JVM, o que define a ordem dos ramos no mapeador de erro.

Do lado iOS, a interop é Kotlin/Native ↔ Swift (interface Kotlin implementada em Swift) e cinterop com `platform.Security`.

Segurança do token

- Nunca em arquivo versionado; `local.properties` é ignorado pelo Git e só pré-preenche a tela, sem login automático. `git grep -n "0t_"` mostra só tokens falsos de teste.
- Gravado só depois de validado pela 1.ª página; erro ou cancelamento na validação apaga o token da sessão.
- Persistência cifrada: Keystore (AES-GCM) no Android, com `allowBackup="false"`; Keychain no iOS, com acesso `ThisDeviceOnly`. Nos dois casos o token não vai para outro aparelho por backup.
- Fora de logs: nenhum `println/Log/NSLog`, nenhum plugin `Logging` do Ktor, token fora de mensagens de exceção. Na tela, só mascarado (`0t_ab...wxyz`).
- Só builds de debug embutem o token de desenvolvimento; o release sai com `GdiBuildConfig.TOKEN = ""` sem parâmetro. Em produção o token ficaria num backend, e o app receberia só URLs de vídeo de curta duração.

Contrato real × Swagger

Medido na conta de teste entre 02 e 05/10/2026; onde diverge do Swagger, vale o medido:

- Toda rota é `POST`, e **toda resposta é JSON com `content-type: text/plain`**. Um HTTP 200 pode trazer `"status":"erro"`; um corpo de erro pode ser uma string JSON (401 `"Não autorizado"`).
- Token inválido → 401; expirado → 403 com mensagem própria; renovar token já expirado → 400. A renovação invalida o token antigo.
- Listagem: `pagina`, `tamanhoPagina` e `origem` são obrigatórios; `pagina` começa em 1; `origem` no pedido é plural (`todos/vinculados/compartilhados`) e no item é singular. Não há total: fim da lista = página com menos itens que o tamanho pedido.
- A API não informa a categoria do dispositivo: é inferida pelo prefixo de modelo/nome.
- Fechadura e bateria exigem `ns` composto `NsDispositivo_NsHub_IdProdutoHub`, com o `idProduto` da própria fechadura no corpo.
- Vídeo: o Swagger promete MP4 por HTTPS, `session_id`, `monitor_url` e cota; a resposta real é **só uma URL RTSP** que expira sozinha. `canalVideo 0` e `1` devolvem o mesmo RTSP; `streamId 0` → 500.
- Volume da fechadura → 500 na fechadura de teste, mesmo com o pedido que a API reconhece (um par `ns/idProduto` errado dá 404, o certo dá 500); estado, bateria e online da mesma fechadura respondem normalmente. Rotas de cota e sessões de streaming → 403 sem plano.
- **Online de subdispositivo** (medido em 05/10/2026 com este app): `/produtos/online/v1` com o `ns` puro da fechadura Zigbee responde `online:false` mesmo com a listagem dizendo `"status":"online"`; com o `ns` composto responde `true`. Câmera e hub respondem certo com o `ns` puro. O app consulta com o `ns` composto só nos subdispositivos.

- `criar-fluxo-video` pode voltar HTTP 500 transitório (1.ª chamada a frio em 05/10/2026); uma nova chamada funciona.

Métricas medidas

Métrica	Valor
<code>criar-fluxo-video</code> (pedido da sessão)	~2,4 s
1.º quadro a quente, Android	~3 s
1.º quadro a frio, Android	~21 s (proxy da nuvem; igual em qualquer cliente)
1.º quadro a frio, iOS	37,8 s (uma única medição, VLCKit por software)
Com este app no Moto G9 Play (05/10/2026)	22,6 s a frio; 3,6 s a quente; 2,3 s ao trocar de lente
Com este app no iPhone 13 Pro (06/10/2026)	39,2 s a frio (VLCKit por software)
Com este app no simulador iOS (05/10/2026)	2,4 s com o proxy já aquecido
APK debug com 3 ABIs	~165 MB (libVLC)

Próximos passos

- iOS: parar o player em background observando `LocalLifecycleOwner`, como no Android, e testar o buffer de 800 ms sem `clock-jitter`.
- Distribuição Android: AAB com ABI splits e sem `x86_64` no release.
- Tela de licenças (LGPL 2.1+ do libVLC/VLCKit) e revisão jurídica antes de publicar, sobretudo no iOS.
- Media3 para o caso HTTPS/MP4, se um dia a API devolver o que o Swagger promete.
- Backend próprio: guardar a credencial, entregar só URLs de vídeo curtas e receber webhooks (eventos em tempo real nas abas de mensagens).
- Divisão em módulos (`:core:domain`, `:data:gdi`, `:feature:home`, `:feature:camera`, `:feature:lock`) quando entrar um segundo parceiro ou uma nova feature.
- `detekt` e `ktlint` no CI.

Apêndices

Decisões

- **Ktor com parse manual.** A GDI serve JSON com `content-type: text/plain`, que o `ContentNegotiation` não converte. A resposta é lida com `bodyAsText()` e `Json.parseToJsonElement`; o parser tolera corpo de erro que é uma string JSON, envelope `"status":"erro"` dentro de HTTP 200 e nomes de campo alternativos. O corpo do pedido é montado com `buildJsonObject`, porque os três campos da listagem são obrigatórios (sem origem a API responde 500).
- **libVLC (Android) e VLCKit (iOS), não Media3 nem AVPlayer.** A API devolve um RTSP de um proxy da nuvem cujo SDP anuncia H.265 e AAC **sem nenhuma linha `a=fmtp`** (os parâmetros do H.265 vêm dentro do fluxo). O RTSP do Media3 recusa esse SDP ("missing attribute fmtp") e o AVPlayer não toca RTSP. O VLC aceita. Ajustes medidos: decodificação por software desde o 1.º pacote (o decoder de hardware precisaria da resolução que viria do `fmtp`), áudio desligado, buffer de 800 ms no Android (`LIVE_NETWORK_CACHING_MS`) e todas as chamadas bloqueantes do libVLC (`stop`, `play`, `release`) numa única thread de controle (`vlc-control`), na ordem em que foram pedidas: nem sair da tela nem ir para background trava a main thread (evita ANR). No iOS o VLCKit fica em Swift, implementando uma interface Kotlin (`NativeVideoPlayer`): inversão de dependência, sem cinterop com o VLCKit.
- **Android Keystore com AES-GCM, direto.** Chave AES-256 que nunca sai do Keystore; `iv:ciphertext` em `SharedPreferences` privadas. A `security-crypto` (`EncryptedSharedPreferences`) foi descontinuada pela Google; o Keystore direto não traz dependência extra. Sem `setUserAuthenticationRequired` (invalidaria a chave ao trocar o bloqueio de tela) e com `allowBackup="false"`.
- **Keychain via platform.Security.** Acesso pelo cinterop que o Kotlin/Native já traz, com `kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly`: o token não migra para outro aparelho por backup, como o `allowBackup="false"` no Android. Os testes iOS usam armazenamento em memória, porque o executável de teste não tem o entitlement do Keychain.
- **Koin, não Hilt.** O Hilt só existe no Android; o Koin roda em `commonMain` e entrega o `ViewModel` com `koinViewModel`.
- **Lista de DeviceProvider.** Cada parceiro implementa `DeviceProvider` (e `LockController`, se tiver fechaduras); o repositório recebe **todos** pelo Koin (`getAll()`), junta as páginas e delega cada operação ao parceiro dono do dispositivo (`Device.providerId`).
- **Renovação do token serializada com Mutex.** Em 401/403 o app renova uma única vez e repete a chamada. Chamadas paralelas fazem uma só renovação ("o token já mudou? então só repete"), porque o token antigo deixa de valer e uma segunda renovação seria recusada. Se a renovação falhar, vale o status original (403 → expirado) e o app pede um token novo.
- **Concorrência na listagem.** `Mutex` para escritas, contador de geração para que a resposta de um filtro antigo nunca apareça, e `Job`s separados para que trocar o filtro cancele em vez de esperar. `CancellationException` é sempre relançada (`resultOf` no lugar de `runCatching`).
- **Nativo por expect / actual de funções e vals de topo**, sem `expect class`, que exigiria flag de compilador.
- **Java na borda Android** (ver "Interoperabilidade com Java"): a regra de negócio fica em Kotlin, no `commonMain`, e o `androidApp` tem código e teste em Java chamando essa regra.

Testes

91 testes Kotlin em 14 classes de `shared/src/commonTest/kotlin/br/com/pompeo/casa/` (meta do projeto: ≥ 70), rodando na JVM e no simulador iOS: os 85 da lista da **especificação**, um de regressão para um fato da API medido no aparelho (`onlineUsesCompositeNsOnlyForSubdevices`) e cinco dos comandos da fechadura no `viewModelScope`. Usam `kotlin.test`, `kotlinx-coroutines-test` e `ktor-client-mock`, sem `MockK`: os dublês são fakes das interfaces do domínio (`TestSupport.kt`) e todo token é obviamente falso. Mais **4 testes**

em Java (JUnit 4) no androidApp , que chamam a regra Kotlin TokenFormat como método estático (@JvmStatic).

```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
./gradlew :shared:testAndroidHostTest      # JVM do host, sem emulador
./gradlew :shared:iosSimulatorArm64Test    # binário de teste no simulador iOS
./gradlew :androidApp:testDebugUnitTest    # testes em Java (JUnit 4)
```

Relatório do Android: shared/build/reports/tests/testAndroidHostTest/index.html . O workflow de CI roda as duas suítes a cada push; hoje o job iOS passa e o job Android falha na preparação do SDK (ver "Limitações conhecidas").

Classe	Testes	O que cobre
TokenFormatTest	3	normalização, plausibilidade e máscara do token
DomainModelTest	4	origem, filtro, hasMore , escala de volume
TokenRepositoryImplTest	7	sessão em memória x persistido, falha do armazenamento seguro
GdiApiTest	19	Bearer, corpo, text/plain , renovação única (inclusive em paralelo), fechadura, vídeo, online de subdispositivo, erros reais
GdiDeviceParserTest	5	envelopes e nomes de campo alternativos, dedup, tipos
GdiRealResponseTest	2	resposta real anonimizada, ns composto, data formatada
GdiErrorMapperTest	5	token ausente/inválido/expirado, rede, timeout, servidor
GdiFormatTest	2	data/hora e tipos de evento da fechadura
DeviceRepositoryImplTest	17	paginação, filtro com fallback, geração, cancelamento, logout, dois parceiros
SubmitTokenUseCaseTest	8	validar antes de gravar, renovação durante a validação, cancelamento
ChangeLockVolumeUseCaseTest	3	grava, relê e assume o pedido se a releitura falhar
HomeViewModelTest	10	validação em viewModelScope , aviso de armazenamento, cancelamento, comandos da fechadura e do volume no viewModelScope (um por vez, erro e volume aplicado)
MiboLogicTest	5	lentes, faixa da semana, rótulos
CameraFormatTest	1	"1º quadro em X s"
TokenFormatJavaTest (Java, androidApp)	4	a mesma regra do token chamada de Java: máscara, plausibilidade, normalização

Limitações conhecidas

- **Comandos físicos nunca executados no equipamento real.** Abrir/fechar a fechadura e mudar o volume agem num aparelho de terceiros; foram validados com a resposta documentada ({"status":"sucesso"} sem data) nos testes. No app, todo comando passa por um diálogo de confirmação.
- **Leitura de volume dá HTTP 500** na fechadura de teste (medido em 03, 05 e 06/10/2026). Em 06/10 foram testadas variações do pedido, só de leitura: com o par certo (ns composto + idProduto da fechadura, o mesmo que funciona para estado, bateria e online) a API reconhece o aparelho e responde 500 "Erro desconhecido" em ~1,4 s, em todas as tentativas; com um par errado (ns puro + idProduto da fechadura, ou ns composto + idProduto do hub) responde 404 "Dispositivo não encontrado". O formato do pedido está certo, e a falha é da GDI para esta fechadura. A tela mostra "Volume indisponível (a API respondeu HTTP 500)" com "Tentar de novo" e mantém a escrita habilitada; depois de gravar, se a releitura falhar, assume o valor pedido.

- **Mesma imagem nas duas lentes** da câmera Dual: a GDI devolve o mesmo RTSP para `canalVideo 0` e `1`. O app envia o canal certo e avisa na tela.
- **Partida a frio do proxy RTSP da nuvem:** cerca de 21 s até o 1.º quadro no Android (igual em qualquer cliente; a quente, cerca de 3 s). No iOS houve uma única medição a frio de 37,8 s. O iOS mantém buffer de 300 ms + `clock-jitter 0`; alinhar com os 800 ms do Android é **hipótese não testada**.
- **Sem eventos em tempo real.** Movimento, campainha e aberturas chegam só por webhook, que exige backend; as abas de mensagens dizem isso.
- **Token em `const val` é extraível.** Um valor de `local.properties` compilado num APK ou binário pode ser lido por quem tiver o arquivo. Aceitável só em demo; por isso só os builds de debug embutem o valor, e o release sai sem ele. Em produção, o token fica num backend e o app recebe só URLs de vídeo de curta duração.
- **Áudio desativado:** o AAC vem sem `config` no SDP e vira ruído. O botão de mudo alterna, mas não tem efeito audível.
- **URL RTSP expira** (`expire`): depois de muito tempo em background a reconexão pode falhar; aparece o erro com "Tentar novamente", que pede uma sessão nova.
- **iOS só para o player no dispose da tela.** O Android para em `ON_STOP` e reconecta em `ON_START`. No iOS o `stop()` do VLCKit roda na main thread (risco de travar ao sair, não observado).
- **APK de cerca de 165 MB** em debug com 3 ABIs, porque o libVLC traz um `.so` grande por ABI.
- **CI:** o workflow roda a cada push; o job iOS passa, e o job Android falha hoje na preparação do SDK no runner (as suítes rodam verdes localmente e nos aparelhos).
- **Licença LGPL 2.1+** do libVLC e do VLCKit: falta tela de licenças, e a distribuição pede revisão jurídica, sobretudo no iOS (linkagem com o app).
- **Tipo de dispositivo inferido** pelo prefixo de modelo/nome (a API não informa categoria). Medido para câmera (`iM`), fechadura (`MFR`) e hub (`MCA / IOT-ZG`); os demais prefixos e parte dos tipos de evento do histórico são hipótese.
- **criar-fluxo-video pode responder HTTP 500 transitório:** em 05/10/2026 a primeira chamada a frio voltou 500; a tela mostrou o erro amigável e "Tentar novamente" (que pede uma sessão nova) abriu o vídeo.
- Rotas de cota e sessões de streaming respondem 403 na conta de teste (sem plano) e não são usadas; sem `session_id` na resposta, não há sessão a encerrar (a URL expira sozinha).

Verificação em aparelho

Estado: executado em 05/10/2026 no Moto G9 Play (Android 11) com a API real e a conta de teste; no iOS, parcialmente (iPhone 13 Pro físico em 06/10/2026: token vencido real, Home e vídeo ao vivo com a API real; fechadura e hub conferidos só no simulador iPhone 17 com a API real). A linha 8 ainda depende do modo avião no aparelho; foi conferida só no simulador, contra um servidor local que imita o contrato da seção "Contrato real x Swagger". Nenhum comando de fechadura ou de volume foi confirmado.

Preparação: apagar os dados do app (`adb shell pm clear br.com.pompeo.casa`; no iPhone, apagar e reinstalar), gerar um token novo no portal e colocá-lo em `local.properties` ou colá-lo na tela. Comandos de fechadura e de volume só são confirmados com autorização explícita do dono da conta.

#	RF	Passos	Deve aparecer	Data/hora	Aparelho	Resultado
1	RF01	Abrir o app sem dados	"Conectar à conta Intelbras", escudo verde, campo "Token de acesso" mascarado; com <code>local.properties</code> , "Pré-preenchido pelo <code>local.properties</code> (desenvolvimento)"	05/10 23:27	Moto G9 Play	OK: igual a <code>app-01</code> ; sem <code>gdi.token</code> , "Entrar" desabilitado; com ele, o aviso de pré-preenchimento

#	RF	Passos	Deve aparecer	Data/hora	Aparelho	Resultado
2	RF04	"Entrar" com um token expirado	"Seu token expirou (ele vale cerca de 2 horas)...", sem sair da tela	06/10 11:27	iPhone 13 Pro	OK com a API real: o token salvo venceu e a Home mostrou "Seu token expirou (ele vale cerca de 2 horas)... (403; a renovação respondeu 400). Na tela de token, conferido só contra o mock
3	RF04	"Entrar" com um texto qualquer de 8 ou mais caracteres	"Token inválido ou expirado. Gere um novo token..."	05/10 23:27	Moto G9 Play	OK: "Token inválido ou expirado..." (a API real respondeu 401 "Não autorizado, verifique os seus limites disponíveis")
4	RF01/RF02	"Entrar" com o token válido	"Validando token..." no botão → Home com a câmera iM4 Dual (badge 2), a fechadura MFR 2030 (via hub) e o hub MCA 1002	05/10 23:33	Moto G9 Play; simulador iOS	OK: iM4 Dual (badge 2), MFR 2030 (via hub), MCA 1002, igual a app-03
5	RF01	Fechar e reabrir o app	Vai direto para a Home (token do Keystore/Keychain)	05/10 23:52	Moto G9 Play	OK: abriu direto na Home
6	RF07	Chips Compartilhados → Vinculados → Todos	Compartilhados: "Nenhum dispositivo encontrado" + "Nenhum dispositivo compartilhado com esta conta."; os outros: 3 dispositivos	05/10 23:53	Moto G9 Play	OK: Compartilhados vazio com o texto do filtro (igual a app-05); Vinculados e Todos com 3
7	RF08	Definições → Itens por página 2 → Início → "Carregar mais"	"1 página(s) · 2 dispositivo(s) · 2 por página" + "Carregar mais" → "2 página(s) · 3 dispositivo(s) · 2 por página" + "Sem mais informações"	05/10 23:54	Moto G9 Play	OK: "1 página(s) · 2 dispositivo(s) · 2 por página" → "2 página(s) · 3 dispositivo(s) · 2 por página" + "Sem mais informações"
8	RF04/RF08	(a) Modo avião → Definições → Atualizar → Início. (b) Sem modo avião, 2 por página e só a 1.ª página → ligar o modo avião → "Carregar mais"	(a) Cartão "Sem conexão com a internet..." com "Tentar novamente"; a lista anterior não some das telas de detalhe. (b) Os 2 itens continuam; abaixo, "Sem conexão..." + "Tentar novamente" (que, sem modo avião, carrega a página 2)	—	Simulador (mock)	Parcial: "Sem conexão..." conferido contra o mock fora do ar; modo avião no aparelho pendente
9	RF03	Abrir a iM4 Dual (a frio)	% e etapa sobem; depois "AO VIVO" e "1º quadro em ~21 s"	05/10 23:34	Moto G9 Play; simulador iOS	OK: % e etapa, "AO VIVO", "1º quadro em 22,6 s" a frio no Android (a 1.ª sessão voltou HTTP 500 transitório; "Tentar novamente" resolveu); iOS com VLCKit: 2,4 s com o proxy já aquecido
10	RF03	Voltar e abrir de novo	"1º quadro em ~3 s"	05/10 23:35	Moto G9 Play	OK: "1º quadro em 3,6 s"; sair da tela sem ANR e sem "late video" no logcat

#	RF	Passos	Deve aparecer	Data/hora	Aparelho	Resultado
11	RF03	"Lente fixa"	Reinicia (~3 s) e mostra "A API GDI devolve o mesmo vídeo para as duas lentes desta câmera."	05/10 23:54	Moto G9 Play	OK: reiniciou em 2,3 s e mostrou a nota das lentes
12	RF03	Tela cheia, mudo, PTZ, gravar, microfone, foto	Tela cheia não reinicia o vídeo; mudo alterna (sem efeito audível: áudio desativado); os demais mostram o Snackbar "Indisponível na API GDI"/"PTZ não disponível na API GDI"	05/10 23:55	Moto G9 Play	OK: tela cheia sem reiniciar o vídeo; PTZ mostra "PTZ não disponível na API GDI"
13	RF05	Abrir a MFR 2030	"Porta fechada", bateria %, Online, "Pressione e segure para abrir a porta."	05/10 23:52	Moto G9 Play	OK após correção: 26 %, Online; a porta estava aberta ("Porta aberta" + "Pressione e segure para trancar a porta."). Antes da correção aparecia "Offline" (ver "Contrato real x Swagger")
14	RF05	Segurar o círculo 600 ms	Diálogo "Destrancar a fechadura?" → Cancelar	05/10 23:56	Moto G9 Play	OK: "Trancar a fechadura?" (porta aberta); fechado com Voltar, nenhum comando enviado
15	RF06	Cartão Volume	"Volume indisponível (a API respondeu HTTP 500)" + "Tentar de novo"; Mudo/Baixo/Médio/Alto habilitados (não tocar sem autorização)	05/10 23:37	Moto G9 Play	OK: "Volume indisponível (a API respondeu HTTP 500)" + "Tentar de novo"; níveis habilitados e não tocados
16	RF09	Histórico	Eventos "Remoto (APP)"/"Por dentro (manual)" com data e hora; "Ver mais" se houver 10	05/10 23:37	Moto G9 Play	OK: 10 eventos reais "Remoto (APP)" com data e hora
17	—	Hub MCA 1002	Online, faixa da semana com hoje em verde, 1 subdispositivo, "Firmware 2.4.628243" com "Atualizado"	05/10 23:37	Moto G9 Play; simulador iOS	OK: Online, faixa da semana com hoje em verde, 1 subdispositivo, "Firmware 2.4.628243" "Atualizado" (igual a app-18)
18	—	Definições → Sair	Volta à tela de token; o "voltar" do sistema não retorna à Home	05/10 23:56	Moto G9 Play	OK: voltou ao token; o "voltar" saiu do app; reabrir pediu o token de novo

#	RF	Passos	Deve aparecer	Data/hora	Aparelho	Resultado
19	—	Repetir 4, 9, 13 e 17 no iPhone	Mesmo comportamento; vídeo via VLCKit (1.º quadro a frio pode passar de 30 s)	06/10 12:43	iPhone 13 Pro; simulador iOS	Parcial. No iPhone físico: 4 OK (iM4 Dual com badge 2, MFR 2030 via hub, MCA 1002; banner e nomes inteiros na tela de 390 pt) e 9 OK ("AO VIVO", "1º quadro em 39,2 s" a frio via VLCKit; o relógio da câmera avançava, e o Moto G9 mostrou a mesma cena no mesmo momento). 13 e 17 conferidos só no simulador; falta repeti-los no iPhone físico

Passo final: capturar cada tela no Android (`adb exec-out screencap -p > tela-<nome>.png`) e no iPhone e comparar lado a lado com as capturas de referência do Mibo Smart. Só são aceitas diferenças de recurso que a GDI não oferece ou de conteúdo de outra conta. Depois, salvar as telas finais em `docs/telas/` (`adb exec-out screencap -p > docs/telas/android-<tela>.png`; no iPhone, botões laterais, salvando como `docs/telas/ios-<tela>.png`) e referenciá-las neste README.

Uso de IA

O app foi construído com um agente de IA de código (Claude Code), em **desenvolvimento orientado por especificação**. Antes do código foi escrita uma especificação com:

- marcos M1–M12 (do esqueleto do projeto à verificação em aparelho), cada um com objetivo, arquivos e um critério de pronto verificável por comando;
- o contrato da API GDI **medido** na conta de teste, com as divergências do Swagger e as armadilhas de plataforma já encontradas;
- a lista dos testes esperados e as capturas de referência do app Mibo Smart para cada tela.

A implementação foi dividida entre agentes por área (domínio e dados, telas, vídeo, testes, documentação), presos a um contrato de nomes e assinaturas, com um agente de build compilando e corrigindo no fim. A verificação não depende da palavra do agente: testes unitários em `commonTest` (JVM e simulador iOS), builds Android e iOS, os `grep` da regra de dependência, o CI no GitHub Actions (a cada push), o roteiro em aparelho real com a API real (no Android físico e parcialmente no iOS) e a comparação lado a lado com as capturas de referência. Comandos físicos na fechadura não são disparados pelo agente: exigem autorização explícita do dono da conta.

A especificação usada está versionada em `docs/sdd/PROMPT-CASE-KMP.md` (só o ID do certificado Apple foi ocultado; as capturas de referência, de terceiros, não estão no repositório). O registro de como ela foi executada e verificada, etapa por etapa, com o que só o aparelho real revelou, está em `docs/sdd/LOG.md`.